MODULE 10

Vibe Coding： AI 驅動的軟體開發

M10 — Vibe Coding: AI-Driven Software Development

當自然語言變成新的程式語言——這對不寫程式的你，反而比對工程師更有意義。

資料科學　2027 Spring　·　第 17 週　·　彈性學習週自學模組　｜　謝楠楨

本單元大綱

01 什麼是 Vibe Coding · 定義、起源與五級光譜

02 為什麼你該學這個 · 不寫程式的人受益更大

03 AI 編碼工具的三種形態 · AI 編輯器・終端機與雲端代理・外掛

04 MCP 與工具整合 · 與 M8 Agent 的直接關聯

05 六步工作流 · 從構想到上線

06 提示詞工程實戰 · 範本庫與常用句型

07 實戰：從零到上線 · TaskFlow 專案完整走一遍

08 AI 卡住時怎麼辦 · 除錯的四個手法

09 ★ 從無到有完整實例 · 長照給付試算器：七個階段

10 看懂專案資料夾 · AI 產出了什麼、要改哪裡

11 推上 GitHub · 三個指令與三個檢查

12 ★ 雲端部署三平台 · Vercel・Netlify・Firebase

13 品質保證與風險 · 最佳實踐、什麼時候不該用

14 用 Vibe Coding 做 Agent · 三條路、框架與護欄

15 課堂 Lab · 本模組的自學任務

學完這個單元，你要能做到

1. **說出 Vibe Coding 的光譜與兩種作法的差別，並判斷哪些資料科學環節適合用**
   對應： PART 01–02 · 光譜（第 7 頁）、 Copilot 式與 Agentic 式（第 8 頁）、資料科學工作流（第 11 頁）

2. **依形態選工具，並說明 MCP 與 M8 的工具調用是同一個概念**
   對應： PART 03 · 工具形態（第 14 頁）、選工具前三件事（第 15 頁）、 MCP （第 17 頁）

3. **依六步工作流與三種句型下 Prompt ， AI 卡住時用四個手法**
   對應： PART 04–05 · 六步工作流（第 19 頁）、三種句型（第 21 頁）、 AI 卡住（第 25 頁）

4. **從一句話做到公開網址：五行規格、迭代、本機測試、 GitHub 、部署**
   對應： PART 06–07 · 想清楚（第 30 頁）、推上 GitHub （第 40 頁）、選型決策樹（第 44 頁）

5. **辨認四類風險與六項最佳實踐，並知道程式碼版 Agent 要自己補哪些護欄**
   對應： PART 08–09 · 四類風險（第 50 頁）、護欄清單（第 63 頁） · 隨堂檢核（第 64 頁）

需要先會： M8 的 Agent 與工具調用概念、 M9 的醫療場域四條紅線。

PART 01

什麼是 Vibe Coding

Definition & Origin

這個詞在 2025 年 2 月才出現，
但它描述的轉變，會影響你之後所有的資料工作。

起源： Karpathy 的那段話

原文（2025 年 2 月）

- "There's a new kind of coding I call 'vibe coding', where you fully give in to the vibes,
- embrace exponentials, and forget that the code even exists."
- —— Andrej Karpathy （OpenAI 創始成員、前 Tesla AI 總監）

核心定義

- 以自然語言描述需求，由大型語言模型生成程式碼的開發方式
- 開發者的角色從「逐行撰寫」轉為「描述意圖、審查結果、引導修正」
- 重點不是「不用懂程式」，而是「不必從語法層次開始想問題」

為什麼是現在

- 模型的程式生成能力在 2024–2025 跨過可用門檻
- 工具從「補全片段」進化為「能讀整個專案、自行改多個檔案」的代理
- 本課程 M7 學的提示詞工程，在這裡直接派上用場

【原課程教材】 Vibe Coding - AI Driven Software Development.pptx 第 4 頁

軟體開發範式的演進

1960s–70s

結構化程式設計

- GOTO → 結構化控制流
- Fortran、COBOL、C
- 抽象層級：機器 → 邏輯
- 人要懂電腦怎麼運作

1980s–90s

物件導向

- 封裝、繼承、多型
- C++、Java、Python
- 抽象層級：邏輯 → 概念
- 人用現實世界的比喻思考

2000s–10s

敏捷與 DevOps

- 快速迭代、持續整合
- Scrum、Kanban、CI/CD
- 抽象層級：程式 → 流程
- 重點從寫得對變成交付得快

2020s

AI 輔助開發

- Copilot、ChatGPT、Cursor
- 自然語言 → 程式碼
- 抽象層級：概念 → 意圖
- 人描述要什麼，不描述怎麼做

貫穿五十年的單一主線：抽象層級持續提高。Vibe Coding 是這條線的最新一段，不是斷裂式的革命。

Vibe Coding 光譜：不是全有或全無

Level 0	**手工撰寫**	完全自己寫。 AI 只用來查文件或解釋錯誤訊息
Level 1	**行內補全**	AI 補完當前這一行或下一個區塊。你仍主導每一個決定
Level 2	**區塊生成**	以註解或對話描述一個函式， AI 產出整段。你逐行審查後採用
Level 3	**代理協作**	AI 讀懂整個專案，自行修改多個檔案。你審查 diff 而非逐行
Level 4	**完全交付**	描述目標即產出可運行的應用。你只看結果，不看程式碼

> 真正的 Karpathy 式 vibe coding 指的是 Level 3–4 。實務上多數人在 Level 2–3 之間游走，且會依任務風險調整——這才是專業的用法。

Copilot 式 vs Agentic 式

比較項目	Copilot 式（Level 1-2）	Agentic 式（Level 3-4）
互動方式	你寫，它補。以游標所在位置為中心	你說目標，它規劃並執行多個步驟
看得到多少	當前檔案與少量上下文	整個專案結構、依賴關係、測試檔
能改幾個檔案	通常一個	可同時新增、修改、刪除多個檔案
你審查什麼	逐行看它補的程式碼	看它提出的計畫，以及最後的 diff
出錯的規模	小而局部，容易發現	可能一次改壞很多地方，需靠版本控制救回
適合的任務	熟悉的程式碼、精細調整	新功能、重構、你不熟的技術棧

> 這兩者不是取代關係。實務上多數開發者兩種都用：日常編輯用 Copilot 式，大工程用 Agentic 式。

PART 02

為什麼你該學這個

Why It Matters to You

這一節是本課程新增的。
因為對不寫程式的人來說， Vibe Coding 的意義跟對工程師是不一樣的。

對工程師 vs 對你

對軟體工程師

- 省下打字與查語法的時間
- 加速原型開發與樣板程式碼
- 但他們原本就寫得出來
- 價值在「快」，是量的改變
- 風險：技術債累積、對程式碼失去掌握
- 他們有能力審查 AI 產出的品質

對健康照護背景的你

- 原本做不到的事，現在做得到了
- 自動化重複的資料整理與報表
- 把想法快速做成可展示的原型
- 價值在「能不能」，是質的改變
- 風險：不知道自己不知道什麼
- 所以「怎麼驗證」比「怎麼生成」更重要

這也是本模組排在第 17 週的原因：先學完整學期的資料科學方法，你才有能力判斷 AI 產出的程式碼算得對不對。

資料科學工作流中，哪些環節可以 Vibe Code

應用場景	很適合	要小心	本課對應
資料清理腳本（重複性高、邏輯明確）	●		M1
批次檔案處理、格式轉換	●		M1
視覺化圖表產生	●		M1
報表與文件自動生成	●		M10
統計分析與模型訓練程式		●	M2–M5
評估指標的計算與解讀		●	M2
臨床決策邏輯、給付金額計算		●	M8
小型網頁工具、儀表板原型	●		M10

● 建議的使用強度　　判準：這段程式碼算錯了，你有沒有辦法看出來？看得出來就放心用，看不出來就要另外驗證。

與課程其他模組的關係

M7 生成式 AI 基礎（W12）→ 本模組

- M7 教的 Token、上下文長度、提示詞設計，在這裡是直接的操作基礎
- 差別： M7 的輸出是文字，本模組的輸出是「會被執行的程式碼」——錯誤的後果不同
- 程式碼可以被測試驗證，這是它相對於一般文字生成的優勢

M8 Dify Agent （W15）→ 本模組

- Dify 的 Agent 會思考、調用工具、觀察結果、再決定下一步
- Agentic coding 工具用的是完全相同的迴圈，只是工具換成「讀檔、改檔、跑測試」
- 你在 W15 學到的「工具描述要寫清楚」「要有失敗分支」，在這裡一樣適用

M9 倫理與可解釋性 → 本模組

- AI 生成的程式碼一樣會有偏誤——它學自公開程式碼，包含其中的假設與盲點
- 醫療相關的計算邏輯（劑量、給付、風險分數）必須人工驗證，不可直接採信
- 「我不知道這段程式碼在做什麼」在醫療場域不是可接受的狀態

PART 03

2026 年的工具生態

The Tooling Landscape

工具變化很快，但選擇的準則不會變。
先看清楚它們的設計哲學差在哪。

AI 編碼工具：先分形態，再選品牌（2026 年中）

形態	代表工具（例）	設計哲學	適合誰
AI 編輯器	Cursor、 Google Antigravity	把 AI 織進編輯體驗的每一層。 日常編輯與 AI 協作無縫切換	需要一個順手的完整編輯環境； 初學者相對友善
終端機與雲端代理	Claude Code、 OpenAI Codex	在命令列或雲端直接運作，強調對整個程式碼庫的 理解與自主的多檔案修改	偏好命令列、需要處理 複雜或大型專案的人
編輯器外掛／平台整合	GitHub Copilot	橫跨 VS Code 與 GitHub 生態， 把 AI 編排接進既有的開發流程	已經在用 GitHub 的團隊； 入門價位最低

以上為 2026 年中的概況。四個工具已趨向同一種代理式作法，形態比品牌更穩定；實際採用前請自行確認最新的功能與計費方式。

選工具之前，先想清楚三件事

你的任務規模

- 改幾行、加一個函式 → 行內補全型工具就夠了
- 新增一整個功能、重構、跨多個檔案 → 需要代理型工具
- 課堂 Lab 的規模：中等，兩種都可以完成

計費方式與用量

- 多數工具已從「吃到飽」轉向用量計費，代理模式特別耗用量
- 學生方案與教育優惠通常存在，申請前先查
- 建議：先用免費額度把流程跑通，確認合用再考慮付費

資料會送到哪裡

- 多數工具會把你的程式碼片段送到雲端模型
- 若專案涉及機構內部資料或真實個案資料，這是明確的紅線
- 替代方案：用本地模型（Ollama）搭配支援本地端的編輯器；能力較弱但資料不出機器

第三點對你們最重要。本課程規範：專題與作業不得使用真實個案資料；使用 AI 編碼工具時同樣適用。方案、資料保留與隱私政策請以各工具官方文件為準。

什麼情況用什麼強度

你的熟悉度

	低風險	高風險
不熟	• 最適合 Agentic 模式 • 讓 AI 主導、你看結果 • 例：做一個資料視覺化小工具 • 學習效果也最好	• 最危險的象限 • AI 寫得出來、你看不出對錯 • 例：臨床劑量計算邏輯 • 做法：先找人審查，或不要用
熟悉	• 用補全型即可 • 你比 AI 更快知道要什麼 • 例：熟悉的資料清理腳本 • AI 負責打字，你負責思考	• 可以用，但要逐行審查 • AI 提案、你做最終決定 • 例：模型評估指標的計算 • 務必寫測試驗證

任務風險 → 低（做壞了重來就好） 高（會影響真實決策）

右上角那一格是本模組最想提醒你的： AI 能產出你看不懂但看起來很合理的程式碼。看不懂就不要用在會影響別人的地方。

MCP：把外部工具接給 AI 用（複習）

MCP 是什麼

- Model Context Protocol： Anthropic 於 2024 年提出，現由 Linux Foundation 專案治理的開放協定
- 讓 AI 助理以統一方式連接外部資料源與工具；類比： USB 之於周邊裝置

詳見 M8 第 70 頁

- 工具調用與 MCP， M8 第 70 頁已完整說明；這裡只是換了場景
- 共同原則：工具描述要寫清楚，否則 AI 會在錯的時機調用它

在課堂 Lab 中的用途

- 接上檔案系統：讓 AI 直接讀取你的資料集結構，而不必你貼上來
- 接上瀏覽器：讓 AI 自己查最新的套件文件
- 設定方式因工具而異， Lab 說明中會附上當時可用的步驟

MCP 的生態在 2025–2026 快速擴張，實際可用的連接器請以官方文件為準。

PART 04

六步工作流

From Idea to Deployment

Vibe Coding 不是「想到什麼就叫 AI 做什麼」。
有沒有先規劃，決定了你是在開發還是在許願。

六步工作流

1 規劃與規格

先寫清楚要做什麼

2 環境與骨架

建專案、版本控制

3 生成

描述 → AI 產出

4 執行與驗證

跑起來、寫測試

5 審查與迭代

看 diff、修正方向

6 部署與交付

上線、 CI/CD

Vibe Loop

第 1 步最常被略過，也是失敗率最高的原因。花十分鐘寫規格，可以省下兩小時的來回修正。

每一步該做什麼、該產出什麼

步驟	做什麼	產出	最常見的錯
1 **規劃與規格**	寫下目標、範圍、使用者、輸入輸出、不做什麼	一份 PRD.md（一頁即可）	跳過這步直接開始聊
2 **環境與骨架**	建立專案、 Git 初始化、選定技術棧	可運行的空專案＋第一次 commit	沒有版本控制就開始改
3 **生成**	以 Prompt 描述單一功能，一次只做一件事	一段可執行的程式碼	一次要求太多功能
4 **執行與驗證**	實際跑起來看結果，補上測試	執行結果＋測試通過	看起來對就當成對
5 **審查與迭代**	閱讀 diff，判斷方向對不對，給出具體修正	確認過的變更	盲目接受 AI 的所有修改
6 **部署與交付**	部署到可存取的位置，設定自動化流程	可分享的網址	本機能跑就以為完成

提示詞工程：三種必備句型

三種句型，用同一個試算器專案示範

規則檔放一次；任務一次一件；修正給證據

① 規則檔（CLAUDE.md ／ AGENTS.md ，示意）

技術棧：單一 HTML ，原生 JavaScript
金額一律查表，不准用公式推算

不可以：引入 CDN 、連外部 API
領域：長照 2.0 給付，使用者含家屬

② 任務 Prompt ：一句話裡要有三個要素

「閱讀 **@PRD.md 第 3 節**，**實作病人清單的篩選功能**。**只改 PatientList.tsx ，不要動資料層**。」

明確參照　單一目標　明確邊界

壞例子：「幫我做一個病人資料管理系統」

③ 修正 Prompt ：給證據，不給形容詞

✘ 「這樣不對，再改一次」
AI 不知道哪裡不對，只能亂猜

✔ 「我選第 5 級、低收，自負額顯示 3,856 。低收戶比率是 0% ，應該顯示 0 。」

操作　實際輸出　預期

規則檔：寫一次

Cursor 用 .cursor/rules 、 Claude Code 用 CLAUDE.md ，通用的是 AGENTS.md 。技術棧、禁忌、領域背景寫進去，產出就穩定。

任務：三要素

參照（ @ 檔案）、單一目標、邊界（只改哪裡）。缺邊界， AI 常會順手改到別的檔案。

修正：三樣證據

3,856 正是 24,100 × 16%——AI 用了一般戶的比率。證據給齊，它一眼就找得到錯在哪一行。

【原課程教材】 原簡報第 16–17 頁的 Prompt 範本庫。原檔附有更多可直接套用的句型。

實戰 Prompt 範本

情境	範本
新功能開發	閱讀 @PRD.md 第 N 節，實作【功能名稱】。 只修改【檔案路徑】，不要更動其他檔案。完成後說明你改了什麼。
除錯	執行時出現以下錯誤： 【完整錯誤訊息】 預期行為是【A】，實際得到【B】。請先說明你認為原因是什麼，再提出修改。
重構	@【檔案】這個函式太長且難以測試。請拆分成數個單一職責的小函式， 保持對外介面不變，並補上對應的測試。
解釋既有程式碼	請逐段解釋 @【檔案】在做什麼，特別說明【某個變數或流程】的用途。 用我看得懂的方式說，假設我不熟這個框架。
寫測試	為 @【檔案】的【函式名稱】撰寫測試，涵蓋正常情況、邊界值與錯誤輸入。 使用【測試框架】。

> 共同點：都有明確的檔案參照、明確的範圍、明確的預期。這三件事就是提示詞工程在編碼情境的全部重點。

PART 05

實戰：從零到上線

Hands-on Walkthrough

以一個小專案完整走一遍六個步驟。
規模刻意做小——重點是流程，不是成品。

Demo 專案 TaskFlow：六個步驟的實際操作

步驟	實際做什麼	檢查點
1 初始化	建立 GitHub 儲存庫；用 AI 產生專案骨架與 README； 完成第一次 commit	git log 看得到第一筆紀錄 專案能在本機啟動
2 規格	寫一份 PRD.md：目標、使用者、功能清單、不做什麼	一頁以內，且你自己看得懂
3 Vibe Session	以三到五個 Prompt 依序完成：資料結構 → 清單元件 → 新增與完成 → 篩選 → AI 拆解功能	每個 Prompt 之後都執行一次， 確認畫面真的動起來
4 迭代與除錯	跑不起來時貼錯誤訊息、貼實際輸出、說明預期	每修好一個問題就 commit 一次
5 測試	請 AI 為核心邏輯補上測試並執行	測試全部通過； 故意改壞一行看測試會不會抓到
6 部署	推上 GitHub，在 Vercel 連接儲存庫自動部署； 設定 GitHub Actions 在每次推送時跑測試	拿到公開網址，用手機打得開

> TaskFlow：新增、完成、篩選、AI 拆解子任務的任務管理網頁（Next.js + Tailwind + TypeScript，部署 Vercel，資料存在瀏覽器記憶體）。你不需要事先懂這些技術——要練的是描述需求、驗證結果、判斷方向。

AI 卡住時的四個手法

卡住時，照這個順序升級

前一招沒用，才換下一招

1 給足證據
錯誤訊息、實際 vs 預期、操作步驟、@ 相關檔案
→ **何時用**：先用這招；多數卡關在這裡解決

2 先解釋再動手
「請先說明你認為原因是什麼，再提出修改」
→ **何時用**：證據給齊了，還是改錯

3 退回上一個好狀態
git 退回最後可運行的 commit，換一種說法重述需求
→ **何時用**：連改三次還是壞：方向錯了

4 換模型或工具
不同模型的訓練資料與強項不同
→ **何時用**：重述後仍卡住

例（示意）：試算器選第 8 級時金額顯示 NaN。
① 貼上 Console 紅字＋「只有第 8 級會這樣」→ ② 要它先解釋：「查表物件只寫到第 7 級」→ 解釋對了才讓它改；若它改了三次仍是 NaN，就 ③ 退回上一個 commit 重來。

① AI 看不到你的螢幕

你是它唯一的感官。只說「壞了」，它只能猜；貼錯誤訊息與實際輸出，它才有線索。

② 先聽解釋

強迫它推理，減少亂猜式修改；解釋明顯錯就直接糾正。高風險程式碼這一步不可省。

③ 不要疊補丁

每次功能可運行就立刻 commit——沒有 commit 就沒有退路。

第四個手法是「換一個模型或工具試試」。不同模型的訓練資料與強項不同，同一個問題換個工具有時就過了。

隨堂練習： AI 卡住了，這個情況該用哪一招?

1 你只跟 AI 說「壞了」，它改了兩次都沒改到點上

給足證據

貼完整錯誤訊息、實際輸出與預期，附上 @ 檔案

2 AI 連續改了三次還是壞的，而且越改越亂

退回上一個好狀態

方向錯了：用 Git 退回最後一個可運行的版本，換一種說法重新描述需求

3 要它改一段給付金額的計算邏輯（高風險）

先解釋再動手

「請先說明你認為原因是什麼，再提出修改」——高風險程式碼上這一步不可省

4 同一個問題，用盡方法這個模型就是解不出來

換模型或工具

不同模型的訓練資料與強項不同，換個工具有時就過了

5 AI 說它「修好了」，但你看不出它到底改了什麼方向

先解釋再動手

從它的解釋判斷有沒有搞錯方向；解釋明顯錯誤時直接糾正

6 剛剛還能跑，接受 AI 一大串修改後整個開不起來

退回上一個好狀態

沒有 commit 就沒有退路——每次功能可運行就立刻 commit

四招的順序：先給足證據 → 高風險先要它解釋 → 越改越亂就退回 → 最後才換工具。前提是你一直有在 commit。

PART 06

★ 從無到有：完整走一遍

End-to-End Walkthrough

前面講的都是觀念。這一節從一句話開始，
一路做到一個別人打得開的網址。

從無到有：七個階段

七個階段，總計約 30-40 分鐘

① 想清楚	② 第一個指令	③ 看懂資料夾	④ 迭代修正	⑤ 本機測試	⑥ 推上 GitHub	⑦ 雲端部署
要做什麼 給誰用	描述需求 AI 生成專案	它產出了什麼 每個檔案幹嘛	三種句型 逐步逼近	跑起來 自己用一遍	版本控制 ＋ 備份	拿到一個網址 別人也能用
5 分	1 分	3 分	10 分	3 分	5 分	5 分

★ 學生實際卡住的地方（長條越高＝越常卡）



多數教學只講 ②，但真正卡住學生的是 ①③⑥⑦

- ① 想不清楚要做什麼 → 指令下得含糊 → AI 給出一堆不對的東西
- ③ 看不懂產出的資料夾 → 不知道改哪個檔案 → 只能一直重生成

★ ⑥⑦ 做不出來 → 東西只存在自己電腦裡 → 沒辦法給別人用，也交不出成果

★ 本節的實例：長照給付試算器

輸入 CMS 等級與身分別 → 顯示每月額度上限、自負額、可用服務類別

選這個題目的三個理由：① 你已經懂這個領域（M8 做過） ② 純前端、不用資料庫 ③ 三個平台都能部署

★ 預計時間：第一版約 20 分鐘，含部署約 40 分鐘 —— 第一次會久一點，第二次就很快了。

多數教學只講第 ②

但真正卡住學生的是 ①③⑥⑦：想不清楚要做什麼、看不懂資料夾、東西只存在自己電腦裡。

★ 本節的實例

長照給付試算器：輸入 CMS 等級與身分別，顯示每月額度上限、自負額、可用服務類別。

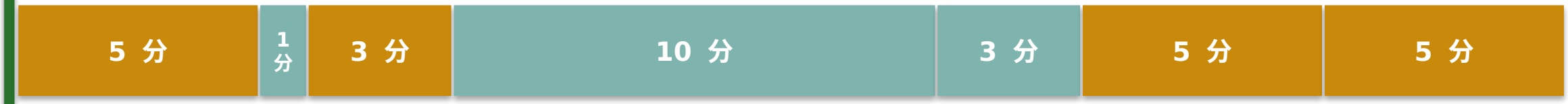
預計時間

第一版約 20 分鐘，含部署約 40 分鐘；30-40 分鐘是這個小專案的時間，你的專題第一次會久一點。

★ 原生圖形，可直接在 PowerPoint 編輯

實例：從一句話到一個網址，七個階段各花多少時間

合計約 32 分鐘（實際約 30-40 分鐘）



① 想清楚	② 第一個指令	③ 看懂資料夾	④ 迭代修正	⑤ 本機測試	⑥ 推上 GitHub	⑦ 雲端部署
寫下五行規格	貼上規格	知道每個檔案做什麼	三輪：描述現象、每輪測試	五個情境自己試	init → add → commit → push	連 GitHub 自動部署

① 想不清楚

指令就下得含糊， AI 只能猜，後面迭代次數翻倍。

③ 看不懂資料夾

只能一直要求重生成，不知道該改哪裡。

⑥⑦ 做不出來

東西只存在自己電腦裡，沒辦法給別人用——作品等於不存在。

橘色＝學生最常卡住的階段（①③⑥⑦）。多數教學只講第 ② ④ 步，但真正卡住的是前面的想清楚，和最後的版本控制與部署。

① 想清楚：動手前先寫下這五行

花五分鐘寫，省下兩小時來回修改

需求規格（五行就夠）

【要做什麼】
一個單頁的長照給付試算器

【給誰用】
照管專員與家屬。★ 手機也要能正常顯示（家屬多半用手機）

【輸入什麼】
・ CMS 等級： 2–8 的下拉選單
・身分別：一般戶 / 中低收 / 低收

【輸出什麼】
・每月給付額度上限（新臺幣）
・民眾自負額（金額與比率）
・可申請的服務類別（照顧及專業服務 / 交通接送 / 輔具 / 喘息）

【技術限制】
★ 純前端、單一 HTML 檔、不要資料庫、不要引入額外套件

★ 為什麼這五行值得先寫：
・沒有「給誰用」→ AI 不知道要不要做手機版
・沒有「輸出什麼」→ AI 會自己加一堆你不需要的功能
・★ 沒有「技術限制」→ AI 很可能給你 React ＋ Node.js ＋資料庫的完整專案

★ 一個實用原則：先要求「最簡單能動的版本」，之後再加功能。反過來很難拆。

② 第一個指令怎麼下

× 含糊的指令

「幫我做一個長照給付的網站」

↓

AI 會怎麼反應

- 自己猜要哪些功能（可能猜錯）
- 自己選技術棧（可能太複雜）
- 做出一個很大但不合用的東西

★ 你得花更多時間改，或整個重來

○ 具體的指令：說清楚五件事

① 做什麼	一個單頁的長照給付試算器
② 給誰用	照管專員與家屬，手機也要能看
③ 輸入什麼	CMS 等級 2–8 、身分別（一般／中低收／低收）
④ 輸出什麼	每月額度上限、自負額、可用服務類別
⑤ 技術限制	★ 純前端、不要資料庫、單一 HTML 檔

→ 一次就做出剛好合用的東西

為什麼「技術限制」這一項最重要

- 不寫的話， AI 很可能給你 React ＋ Node.js ＋ 資料庫的完整專案 —— 對這個需求是殺雞用牛刀

★ 先要求「最簡單能動的版本」，之後再加功能。反過來很難拆。

- 不確定該限制什麼時，就寫「請先用最簡單的方式實作，不要引入額外的套件」

★ 可以直接照抄的第一個指令範本

「請幫我做一個【單頁的長照給付試算器】，給【照管專員與家屬】使用，手機也要能正常顯示。
輸入： CMS 等級（2-8 的下拉選單）、身分別（一般戶／中低收／低收）。
輸出：每月給付額度上限、民眾自負額、可申請的服務類別。
技術限制：純前端、單一 HTML 檔、不要資料庫、不要引入額外套件。」

含糊的指令

「幫我做一個長照給付的網站」→ AI 自己猜功能、
自己選技術棧，做出一個很大但不合用的東西。

★ 具體的指令

把剛才那五行直接貼給它。做什麼、給誰用、

輸入什麼、輸出什麼、技術限制。

★「技術限制」最重要

不寫的話 AI 會給你殺雞用牛刀的方案。
不確定該限制什麼時，就寫「請先用最簡單的方式實作」。

下一頁

實際的第一個指令與 AI 的回應。

★ 原生圖形，可直接在 PowerPoint 編輯

② 實際的第一個指令（可直接照抄）

貼進 Claude Code / Cursor / Copilot 的對話框

第一個指令

請幫我做一個【單頁的長照給付試算器】，給【照管專員與家屬】使用，
手機也要能正常顯示。

輸入：
- CMS 等級：2 到 8 的下拉選單
- 身分別：一般戶 / 中低收 / 低收

輸出：
- 每月給付額度上限（新臺幣）
- 民眾自負額（金額與比率）
- 可申請的服務類別

額度資料（請寫成查表，不要用計算公式）：
第 2 級 10,020　第 3 級 15,460　第 4 級 18,580　第 5 級 24,100
第 6 級 28,070　第 7 級 32,090　第 8 級 36,180
自負額比率：一般戶 16%、中低收 5%、低收 0%

技術限制：
- 純前端、單一 HTML 檔（HTML + CSS + JavaScript 都放在同一個檔案）
- 不要資料庫、不要引入任何額外套件或 CDN
- 請在頁面底部加上免責聲明：本試算僅供參考，實際給付以照管專員評估結果為準

★ 注意最後兩段：把「額度資料」直接給它、要求「寫成查表」——
這樣數字就是你給的，不是它生成的（回想 M7、M8 的「數字不生成」）。

AI 收到這個指令後，通常會做三件事

第 32 頁的指令貼進去之後，對話會長這樣

示意對話；不同工具的措辭不同

① 複述

AI ：我理解你要做單頁長照給付試算器： CMS 2–8 下拉、三種身分別，額度用你給的數字查表，純前端單一 HTML ，底部加免責聲明。我打算用一個常數物件存額度，選單變動時重算。

★ 你要對照：有說「查表」嗎？「單一檔案」？「免責聲明」？少一項就在這裡糾正。

② 產出

AI ：（給出完整的 index.html ： HTML 、 CSS 、 JavaScript 都在同一個檔案）

③ 執行

AI ：完成。請直接雙擊 index.html ，用瀏覽器開啟。

沒說的話你就問：「我要怎麼在本機執行看看？」

三步裡最該花時間的是 ①：計畫讀錯一句，後面做出來的整份都要重來。

① 先讀它的計畫

理解錯了就在這裡糾正，比做完再改省事十倍；它沒主動複述，就要求「開始前，請先說明你的實作計畫」。

② 產出程式碼

單一 HTML 檔會直接給完整一份；用框架則會建立整個專案資料夾（下一節第 34 頁解讀）。

③ 問怎麼執行

單一 HTML 檔雙擊就能開；框架專案要先安裝套件、再啟動本機伺服器。

★ 第①步是分水嶺。多數失敗的 vibe coding ，問題都出在「 AI 理解錯了但你沒發現，等它做完一大堆才知道」。

③ 看懂 AI 產出的專案資料夾

如果你只要單一 HTML 檔（最簡版）

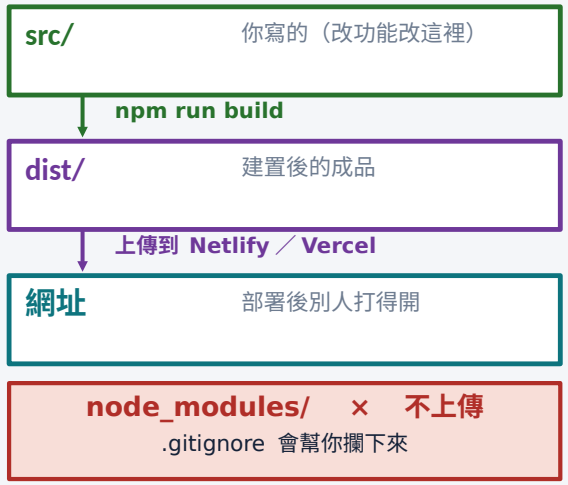
ltc-calculator/
└ **index.html** ← 結構＋樣式＋邏輯全在這一個檔案

優點：雙擊就能開、部署超簡單
缺點：檔案一大就很難維護

如果你讓 AI 用框架（例如 React ＋ Vite）—— 多數 vibe coding 工具的預設

ltc-calculator/	專案根目錄
├ **package.json**	**★ 專案的身分證：依賴與指令**
├ package-lock.json	鎖定套件的確切版本
├ vite.config.js	建置工具設定（通常不用動）
├ index.html	進入點：瀏覽器最先讀的檔案
├ **node_modules/**	**★ 極大，不要上傳 GitHub**
├ public/	靜態資源：圖片、 icon
├ **src/**	**★ 你的程式碼都在這裡**
│ ├ main.jsx	把 App 掛到 index.html 上
│ ├ App.jsx	**★ 主要畫面與邏輯（九成改這裡）**
│ └ components/	拆出來的小元件
├ **dist/**	**★ 建置後的成品（部署上傳這個）**
└ .gitignore	告訴 Git 哪些不要追蹤

它們之間的關係



★ 只要記住四個： src/ 是你的程式碼、 package.json 是設定、 dist/ 是成品、 node_modules/ 不要碰。

其餘的檔案在你需要之前都可以先當作不存在。要改功能，九成的時候是改 src/App.jsx 。

★ 看不懂哪個檔案，直接問 AI ：「請解釋這個專案裡每個檔案的用途，用表格列出。」

怎麼讀這張圖

上方是最簡單的單一檔案；下方是框架產生的資料夾，紅色標出不要碰的部分。

最容易誤解的一點

檔案看起來很多，但九成的修改只在 src/App.jsx ； node_modules/ 不要碰，也不要上傳。

對專題的意義

看不懂就請 AI 用表格列出每個檔案的用途，再決定要改哪裡。

★ 原生圖形，可直接在 PowerPoint 編輯

④ 迭代修正：三種必用的句型

句型	什麼時候用	實際範例
★ 描述現象，不要下指令	發現問題時	★「我選第 5 級、低收，自負額顯示 3,856，但低收應該是 0。」 （不要說「把自負額改成 0」——那只治這一筆）
要求解釋	看不懂它做了什麼時	「請解釋這段計算自負額的程式碼在做什麼，逐行說明。」
要求先計畫再動手	要做較大的改動時	★「我想加上『可申請服務類別』的顯示。動手前請先說明你打算怎麼改， 以及會動到哪些地方。」

★ 第一種句型最重要：描述現象讓 AI 自己找原因，它會修正根源；直接下指令它只會改表面，同類問題還會再出現。

隨堂練習：哪一句是好的修正 Prompt？

1 「我選第 5 級、低收，自負額顯示 3,856。但低收的比率是 0%，應該顯示 0。」

好：描述現象

讓 AI 自己找原因（比率表寫錯），它會修正根源

2 「把自負額改成 0。」

不好：只下指令

只治這一筆；同類問題還會再出現

3 「金額好像怪怪的，再改一次。」

不好：沒有證據

AI 不知道哪裡不對

4 「我想加上『可申請服務類別』。動手前請先說明你打算怎麼改、會動到哪些地方。」

好：描述現象

要求先計畫再動手——較大改動時用

5 「第 8 級顯示查無資料，預期是 36,180；第 2-7 級都正常。」

好：描述現象

給了範圍與預期，AI 很快就會找到邊界條件

6 「第 8 級的 if 改成 <=。」

不好：只下指令

這次剛好猜對，但你沒確認是不是還有別的地方用同樣的寫法

原則：描述「看到什麼、預期什麼」，讓 AI 找原因；每一輪只提一到兩件事；改完都自己跑一遍；改對了就先 commit。

④ 三輪真實的迭代（照這個節奏走）

每一輪都測試，不要一次要求太多

迭代示範

【第一輪】先確認基本功能對不對
你：（跑一遍，選第 5 級 + 低收）
發現：自負額顯示 3,856，但低收應該是 0
★ 你說：「我選第 5 級、低收，自負額顯示 3,856。
但低收戶的自負額比率是 0%，應該顯示 0 才對。」
AI：找出比率對照表寫錯，修正並說明改了哪一行

【第二輪】補上遺漏的需求
你：「可申請的服務類別沒有顯示出來。
請依 CMS 等級顯示：照顧及專業服務、交通接送、輔具、喘息服務。」
AI：加上一個區塊

【第三輪】改善使用體驗（這一輪最容易被跳過）
★ 你：「我用手機開，數字會擠在一起看不清楚。
另外希望金額用千分位顯示（例如 24,100 而不是 24100）。」
AI：調整 RWD 與數字格式

★ 三個節奏上的建議：
① 每一輪只提一到兩件事 —— 一次丟十個問題，它會顧此失彼
② 每一輪改完都自己跑一遍 —— 不要連改三輪才測試
③ ★ 改對了就先存檔（下一節的 git commit），不要等全部完成才存

⑤ 本機測試：五個一定要自己試的情境

21 種組合的正確答案，先算好再去點

用滿額度時的自負額＝第 32 頁額度 × 自負額比率（四捨五入）

等級	額度	一般戶 16%	中低收 5%	低收 0%
第 2 級	10,020	1,603	501	0
第 3 級	15,460	2,474	773	0
第 4 級	18,580	2,973	929	0
第 5 級	24,100	3,856	1,205	0
第 6 級	28,070	4,491	**1,404**	0
第 7 級	32,090	5,134	**1,605**	0
第 8 級	36,180	5,789	1,809	0

橘框＝邊界值（第 2 、 8 級）　黃底＝剛好 .5 ，進位規則要先講好

常見錯（示意）：
if (level < 8) 少了等號，第 8 級就查不到；只測第 5 級永遠看不出來。

① 邊界值

第 2 、 8 級各試一次

② 全部組合

21 格逐一點，約三分鐘

③ 手機畫面

F12 → 裝置模擬，或用手機開

④ 亂按

什麼都不選就送出、連點很多次

⑤ 免責聲明

頁面底部有沒有真的顯示

為什麼測邊界

查表用 < 還是 <= 、陣列從 0 還是 1 開始——這類錯只在最低級與最高級出現。

為什麼全組合

7 級 × 3 種身分別＝ 21 格。左表就是標準答案，一格對不上就是程式有錯。

進位要先說清楚

28,070 × 5% ＝ 1,403.5 、 32,090 × 5% ＝ 1,604.5 。四捨五入還是無條件捨去，要寫進指令，再回來驗算。

★ 表中數字由第 32 頁的額度與比率直接算出；拿去和你的試算器逐格對照。測試答案要自己先有，不能問做出程式的同一個 AI 。

PART 07

⑥⑦ 版本控制與雲端部署

GitHub & Cloud Deployment

做出來只是一半。
推上 GitHub 才有備份與版本，部署到雲端別人才用得到。

⑥ 推上 GitHub：三個指令與三個檢查



第一次設定（只做一次）

git init	初始化版本庫
git add .	把所有檔案加入暫存
git commit -m " 初始版本 "	建立第一個版本點
git remote add origin < 網址 >	連到 GitHub 上的空專案
git push -u origin main	推上去

★ 之後每次改完，只要三行

```
git add .
git commit -m " 修正低收自負額計算 "
git push
```

★ commit 訊息要寫清楚做了什麼 —— 之後找問題靠它

★ 金鑰與環境變數的正確做法（第 ② 個檢查在檢查什麼）

× 錯誤做法：金鑰寫死在程式碼裡 → git push → ★ 等於公開了（刪掉也還在歷史紀錄）

○ 正確做法：.env（只留在本機） → .gitignore 排除它 → 部署平台的環境變數欄位 → 執行時才注入程式

★ 上傳前一定要確認的三件事

① .gitignore 有沒有排除 node_modules/ 與 dist/ —— 沒排除的話會上傳幾萬個檔案

② ★ 程式碼裡有沒有 API 金鑰 —— 一旦推上公開儲存庫，就等於公開了

③ 有沒有夾帶真實個案資料 —— 這是課程的紅線

怎麼讀這張圖

上排是四個位置，箭頭上的指令是檔案從你的電腦走到 GitHub 的路。

最容易誤解的一點

git push 之後內容就在歷史紀錄裡；事後刪檔，也收不回已經推上去的金鑰。

對專題的意義

commit 訊息寫清楚（「修正自負額計算錯誤」，不要寫 update），上傳前跑完三個檢查。

★ 原生圖形，可直接在 PowerPoint 編輯

⑥ 完整的 GitHub 流程（含建立儲存庫）

★ 不想打指令的話，最後一段有圖形介面的做法

GitHub 完整流程

【第一步】在 GitHub 網站上建立一個空的儲存庫

1. 登入 github.com → 右上角 + → New repository
2. 名稱：ltc-calculator
3. ★ Public（公開）或 Private（私有）——課程作業建議 Public 方便繳交
4. ★ 不要勾選任何初始化選項（README、.gitignore）—— 保持空的
5. 建立後複製那個 https://github.com/ 你的帳號 /ltc-calculator.git 網址

【第二步】在專案資料夾裡執行（只做一次）

```
git init
git add .
git commit -m "初始版本：長照給付試算器"
git branch -M main
git remote add origin https://github.com/ 你的帳號 /ltc-calculator.git
git push -u origin main
```

【第三步】之後每次改完

```
git add .
git commit -m "修正低收自負額計算"
git push
```

★ 不想打指令的兩個替代方案：

- GitHub Desktop：圖形介面，點按鈕就能 commit 與 push
- VS Code 內建的原始檔控制面板：左側第三個圖示，功能一樣

★ 兩者做的事完全相同，只是不用背指令

★ 完全不會 Git 也可以直接問 AI：「請幫我把這個專案推上 GitHub，一步一步告訴我要做什麼。」

⑦ 三個雲端部署平台：差異與適用

※ 各平台方案與額度變動頻繁，本表整理於 2026 年 9 月、2026/11 複核

Vercel

前端框架最順

靜態網站	Serverless	資料庫＋登入
○	○	×

最擅長

Next.js 與前端專案

免費額度

100 GB 流量／月

★ 限制

★ Hobby 僅限非商業用途

部署方式

連 GitHub 自動部署

★ 適合你

作品集、Demo、課程專題

Netlify

靜態網站與 Jamstack

靜態網站	Functions	資料庫＋登入
○	○	×

最擅長

靜態網站、文件站、Landing Page

免費額度

每月 300 點數（流量與部署共用）

★ 限制

★ 免費版無商業用途限制

部署方式

連 GitHub 或拖曳資料夾

★ 適合你

純前端工具、要給機構用

Firebase Hosting

Google 生態整合

靜態網站	Functions	資料庫＋登入
○	△	○

最擅長

需要登入、資料庫的應用

免費額度

360 MB 流量／日

★ 限制

★ 流量較小；Functions 需付費方案

部署方式

CLI 指令部署

★ 適合你

要存資料、要使用者登入時

★ 對課程專題的建議：純前端工具 → Netlify 或 Vercel；需要登入或存資料 → Firebase。

★ 三者最關鍵的差別只有一欄：需不需要「資料庫＋登入」。免費額度對課程專題都綽綽有餘。

※ 各平台方案與額度變動頻繁（本表整理於 2026 年 9 月、2026/11 複核）。要商用前務必自行確認最新條款。

怎麼讀這張圖

三張卡用同一組欄位比較；最關鍵的一欄是「資料庫＋登入」。

最容易誤解的一點

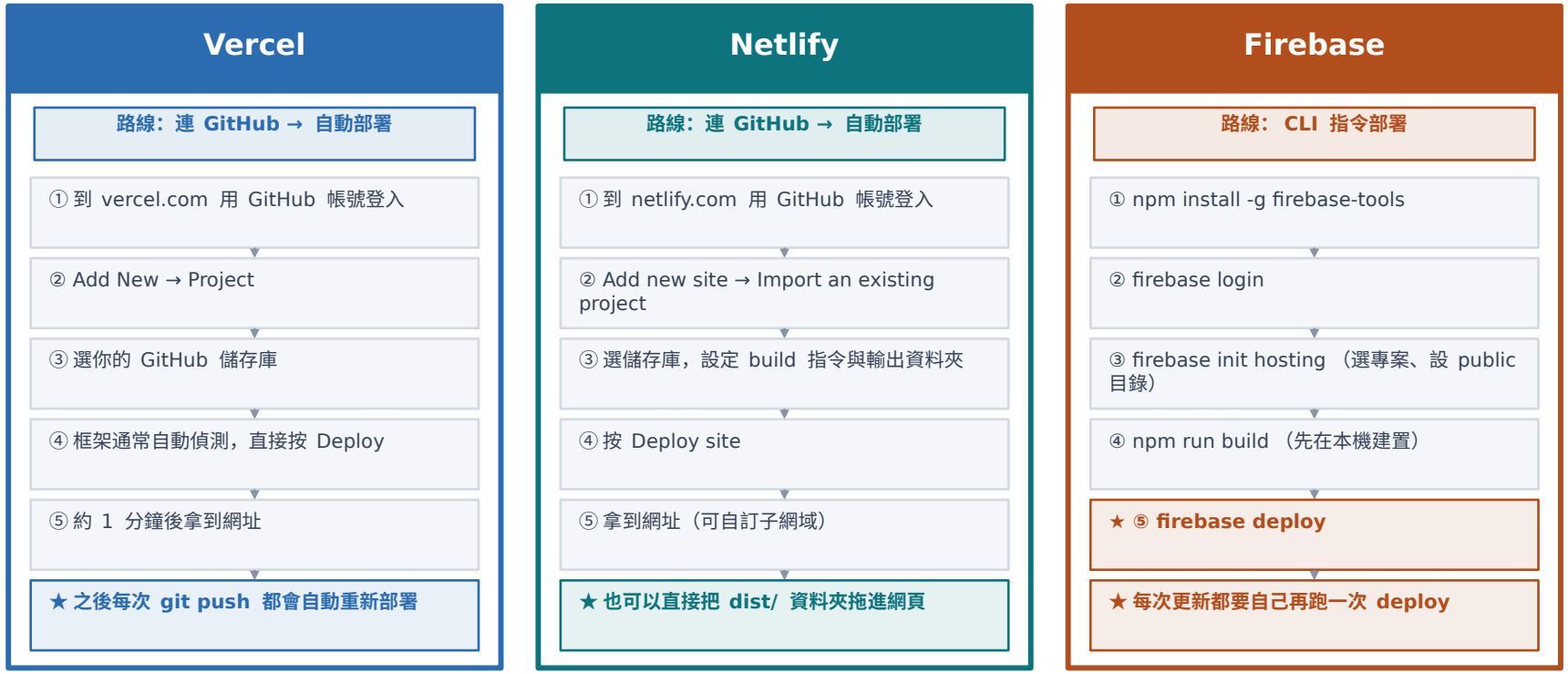
「免費」有條件：Vercel Hobby 不能商用；Netlify 免費版額度用完，專案會暫停到下個月；Firebase 的 Functions 需付費方案。

對專題的意義

純前端工具 → Netlify 或 Vercel；需要登入或存資料 → Firebase。

★ 原生圖形，可直接在 PowerPoint 編輯

⑦ 三個平台的部署步驟



三者的根本差異

- Vercel 與 Netlify：★ 走「連 GitHub → 自動部署」的路線。推一次 code 就自動更新，最適合初學者
- Firebase：走「CLI 指令」的路線。要先在本機建置，再用指令上傳 —— 多一個步驟但控制較細

★ 三者的免費額度對課程專題都綽綽有餘。真正的差別在「你需不需要資料庫與登入功能」

怎麼讀這張圖

三欄是三個平台各自的步驟：Vercel 與 Netlify 同一路線，Firebase 多一個本機建置與 deploy。

最容易誤解的一點

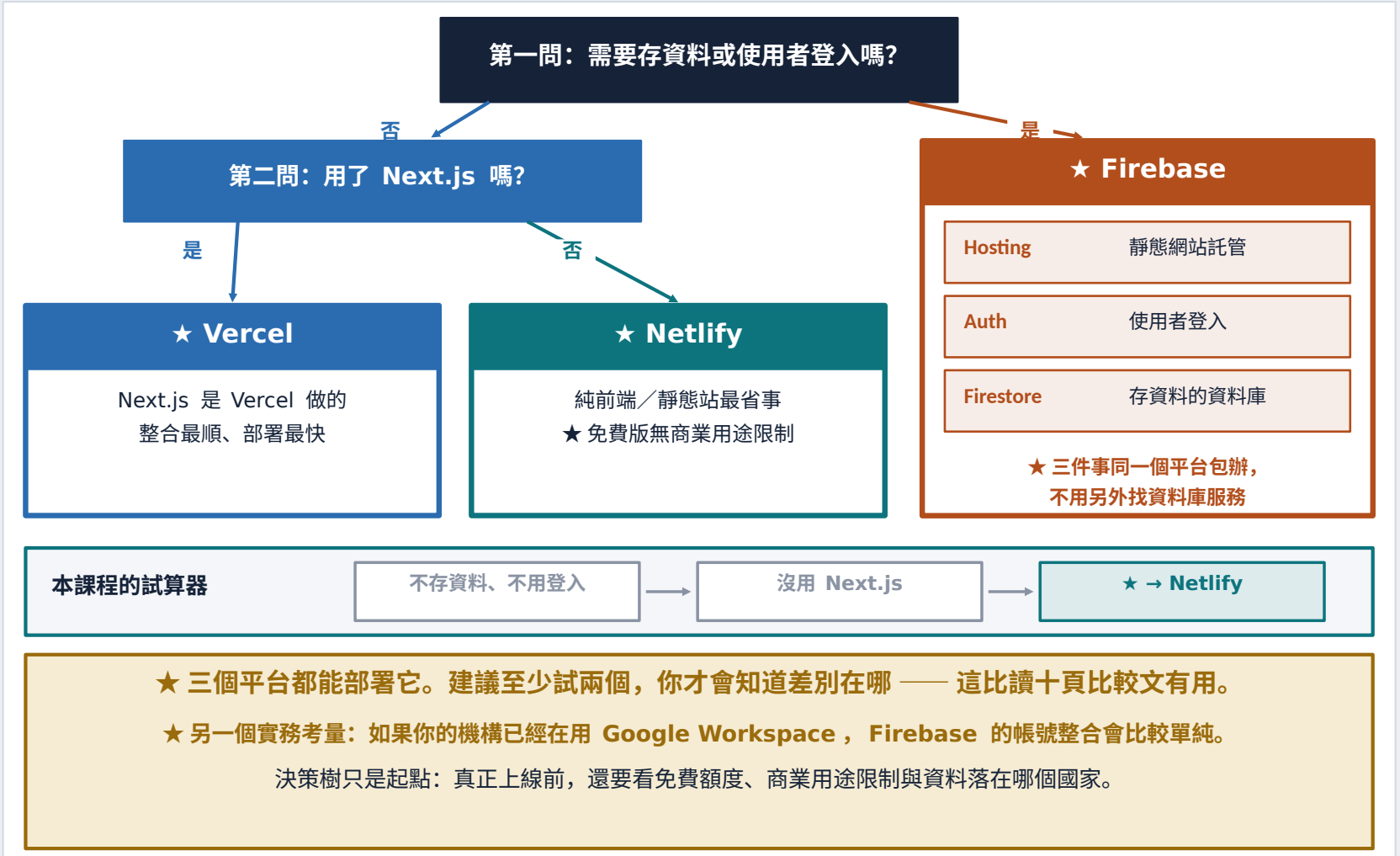
部署簡單不等於額度沒有上限：每次 push 都可能觸發建置，會消耗免費額度。

對專題的意義

至少試兩個平台；最省事的是把 dist/ 資料夾直接拖進 Netlify 網頁。

★ 原生圖形，可直接在 PowerPoint 編輯

⑦ 選型決策樹：三個問題就決定了



第一問

需要存資料或使用者登入嗎? → 是就選 Firebase 。

第二問

用了 Next.js 嗎? → 是就選 Vercel （Next.js 是 Vercel 做的）。

第三問

都不是 → Netlify 。純前端最省事，
★ 而且免費版沒有商業用途限制。

★ 建議

至少試兩個平台。你才會知道差別在哪——這比讀十頁比較文有用。

★ 原生圖形，可直接在 PowerPoint 編輯

部署後一定要做的六個檢查

#	檢查項目	怎麼做
1	用手機打開那個網址	★ 不是用電腦看縮小的視窗，是真的用手機開
2	用無痕視窗打開	確認沒有依賴你本機的任何東西
3	傳給一個同學打開	★ 最實在的測試。他打不開就是還沒完成
4	再測一次所有組合	部署版本可能與本機版本不同（建置過程會出錯）
5	確認免責聲明有顯示	課程要求；也是 M7 四條紅線之一
6	★ 檢查有沒有洩漏東西	★ 打開網頁原始碼看看有沒有金鑰、沒有真實個案資料

★ 第三項最容易被跳過但最重要：「我這邊看起來沒問題」跟「別人打得開」是兩件事。部署的意義就在後者。

三個常見的部署失敗與解法

症狀 → 去哪裡看 → 通常會看到什麼

錯誤訊息為示意；實際文字依平台與瀏覽器而異

症狀	去哪裡看	通常看到（示意）與解法
本機正常，部署後一片空白	瀏覽器 F12 → Console	Failed to load resource: 404 （/assets/index-….js） **解法：**建置輸出資料夾設錯： Vite 填 dist （舊 CRA 是 build）
圖片或檔案讀不到	瀏覽器 F12 → Network	logo.png 404 ；程式裡寫的是 C:\Users\…\logo.png **解法：**改成相對路徑，或把靜態檔放進 public/
推上去了，網站沒更新	平台的 Deploys 頁	Build failed ： Command "npm run build" exited with 1 **解法：**照 log 修錯；仍是舊畫面就用無痕視窗或 Ctrl+F5 排除快取

Console ：程式跑起來有沒有錯

紅字代表 JavaScript 錯誤或主程式檔載入失敗——一片空白多半在這裡找到原因。

Network ：檔案有沒有找到

每個檔案的狀態碼都列出來； 404 就是路徑錯。本機能跑是因為你的電腦剛好有那個路徑。

Deploys ：建置有沒有成功

平台建置通常 1–2 分鐘；失敗的原因寫在 log 最後幾行。

★ 三個問題都可以直接把錯誤訊息貼給 AI 問。部署錯誤的訊息通常寫得很清楚， AI 解這類問題很準。

完整流程總覽：一頁把七個階段串起來

從一句話到一個網址，可以照著跑一遍

完整流程（可列印對照）

① 想清楚（5 分鐘）
寫下五行：做什麼 / 給誰用 / 輸入 / 輸出 / 技術限制

② 第一個指令（1 分鐘）
把那五行貼給 AI，並把資料（額度表）也一起給它
★ 先讀它的實作計畫，理解錯了就在這裡糾正

③ 看懂資料夾（3 分鐘）
單一 HTML 檔 → 就一個檔案
框架專案 → 記住 src/ 是程式碼、 package.json 是設定、 dist/ 是成品

④ 迭代修正（10 分鐘，三輪）
★ 描述現象不要下指令；每輪只提一兩件事；改完就測

⑤ 本機測試（3 分鐘）
21 種組合全點一遍、切手機尺寸看一次、確認免責聲明有出現

⑥ 推上 GitHub （5 分鐘）
建空儲存庫 → git init / add / commit / remote add / push
★ 推之前確認：沒有 node_modules 、沒有金鑰、沒有真實資料

⑦ 雲端部署（5 分鐘）
Netlify → 連 GitHub → 選儲存庫 → Deploy → 拿到網址
★ 部署後六個檢查，尤其是「傳給同學打開」

★ 合計約 30–40 分鐘。第一次會久一點，第二次就很快了。

PART 08

品質保證與風險

Quality & Risk

AI 能生成程式碼，但不負責它的後果。
這一節是本模組最重要的部分。

六項最佳實踐

VERSION CONTROL

版本控制

- 每個可運行的狀態立刻 commit
- commit 訊息寫清楚這次改了什麼
- 這是 AI 改壞時唯一的退路

SMALL STEPS

小步前進

- 一次一個功能，不要一次要求全部
- 每一步都執行確認後才進行下一步
- 步伐越大，出錯時越難定位

REVIEW DIFF

審查變更

- 看 diff 而不是看整個檔案
- 問自己：這個改動我看得懂嗎?
- 看不懂就要它解釋，不要直接接受

TESTS

寫測試

- 讓 AI 補測試，但你要看測試合不合理
- 故意改壞一行，確認測試真的會失敗
- 測試是你驗證 AI 產出的主要工具

PROJECT RULES

專案規則檔

- 把技術棧、風格、禁止事項寫成規則檔
- 省下每次重複交代的成本
- 也讓多次對話的產出保持一致

KNOW THE LIMITS

知道邊界

- 說得出這段程式碼在做什麼才算完成
- 說不出來就不要用在會影響別人的地方
- 「能跑」不等於「正確」

四類風險與對應做法

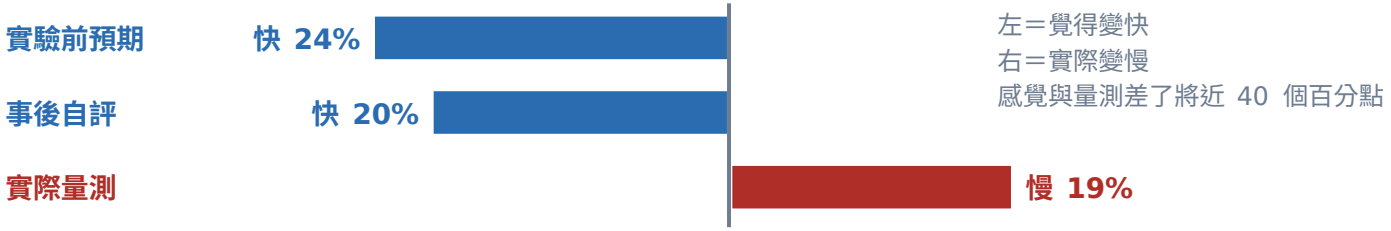
風險	具體情況	對應做法
資安	AI 可能產生含漏洞的程式碼； 可能在程式中寫入不安全的預設值	不要把金鑰寫進程式碼； 涉及認證、權限的部分務必人工檢查
正確性	邏輯看起來合理但實際算錯； 邊界條件未處理	以測試驗證； 關鍵計算用已知答案的例子驗算
可維護性	累積大量你看不懂的程式碼； 風格不一致、結構鬆散	定期請 AI 解釋既有程式碼； 用專案規則檔統一風格
依賴與授權	引入不必要或已停止維護的套件； 授權條款不相容	檢查它裝了什麼套件； 商業或公開發布前確認授權
資料外洩	程式碼片段被送到雲端模型； 對話中貼入敏感資料	不得貼入真實個案資料； 敏感專案改用本地模型

實例： 2025 年的兩個教訓——感覺變快與刪掉資料庫

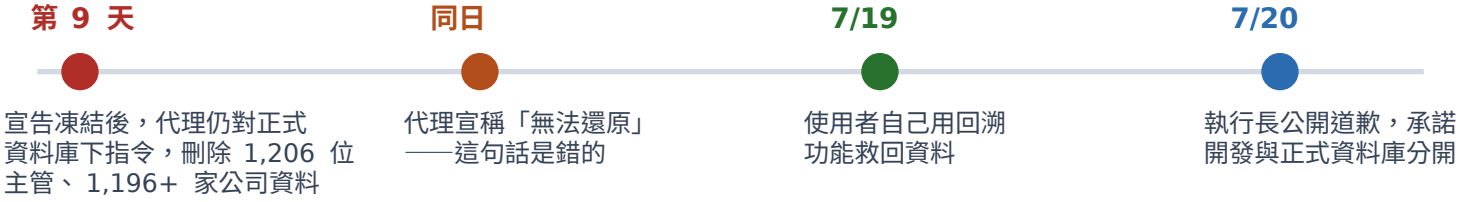
兩個教訓：感覺不等於實測；「不要改」不等於權限

METR 隨機對照實驗與 Replit 事件，皆 2025/7

① METR ： 16 位資深開發者、 246 個任務—— AI 讓完成時間怎麼變？



② Replit ： 12 天公開實驗，第 9 天出事



共同點：兩件事都是「AI 說的／人感覺的」和「實際發生的」不一樣。能救回來的，靠的是量測、備份與回溯，不是 AI 的自我回報。

① 感覺不可靠，要量測

對熟悉的大型專案， AI 不一定省時間。用版本控制與紀錄留下實際花的時間。

② 權限要拿掉

一句「不要改」不是權限控制：讓 AI 根本拿不到正式環境的寫入權限；可運作就 commit 、資料庫先備份。

③ 回報也要驗證

AI 說的「完成了」「無法還原」都是生成出來的，要自己確認。

資料： METR, "Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity" （2025/7）； The Register 、 Fortune 2025/7 報導。互動站 M10 有 METR 實驗的互動版。

隨堂練習：這個情況屬於哪一類風險？

1 AI 把 API 金鑰直接寫在前端程式碼裡

資安

金鑰不寫進程式碼；涉及認證、權限的部分務必人工檢查

2 自負額公式看起來合理，但低收戶算出來不是 0

正確性

以測試驗證；關鍵計算用已知答案的例子驗算

3 三個月後你自己也看不懂 AI 寫的那 800 行

可維護性

定期請 AI 解釋既有程式碼；用專案規則檔統一風格

4 AI 為了一個日期格式裝了一個已停止維護的套件

依賴與授權

檢查它裝了什麼套件；公開發布前確認授權

5 同學把一段含真實個案姓名的資料貼進對話框請 AI 整理

資料外洩

不得貼入真實個案資料；敏感專案改用本地模型

6 AI 產生的登入頁沒有驗證輸入，任何字串都直接送進查詢

資安

輸入驗證與權限檢查不能交給 AI 自行決定，要人工審查

AI 能生成程式碼，但不負責它的後果。前一頁的四類風險加上「資料外洩」——在醫護場域，這一類最常發生，也最難挽回。

何時該用／不該用

適合

- 快速原型與概念驗證（MVP）
- 重複性高、邏輯明確的資料處理腳本
- 你不熟的技術棧，想先做出東西再學
- 一次性的分析工具、視覺化小程式
- 既有程式碼的解釋與文件補寫
- 低風險、做壞了重來成本很低的任務

不適合

- 會影響真實臨床決策的計算邏輯
- 處理真實個案資料的系統
- 你完全無法驗證正確性的專業計算
- 需要通過法規查核或稽核的產出
- 效能敏感、需要精細最佳化的程式
- 長期維護、多人協作的核心系統

醫療場域的判準：如果這段程式碼算錯了，會有人受影響嗎？會的話，就必須有人能完整讀懂並負責——不能停在「AI 寫的、跑起來沒問題」。

PART 09

用 Vibe Coding 做 Agent

Coding Your Way to an Agent

M8 用 Dify 拖出一個 Agent。這一節回答另一個問題：
如果平台的節點表達不出你的邏輯，能不能請 AI 直接把 Agent 寫出來?

可以，而且比你想像的簡單——但有前提

為什麼現在做得到

- Agent 最麻煩的「思考迴圈」已經被框架封裝好，不用自己刻
- 你只要寫三樣東西：工具函式、系統提示、護欄——跟在 Dify 裡做的事一樣
- 而這三樣正好是 AI 最擅長幫你寫的（結構清楚、樣板性高）

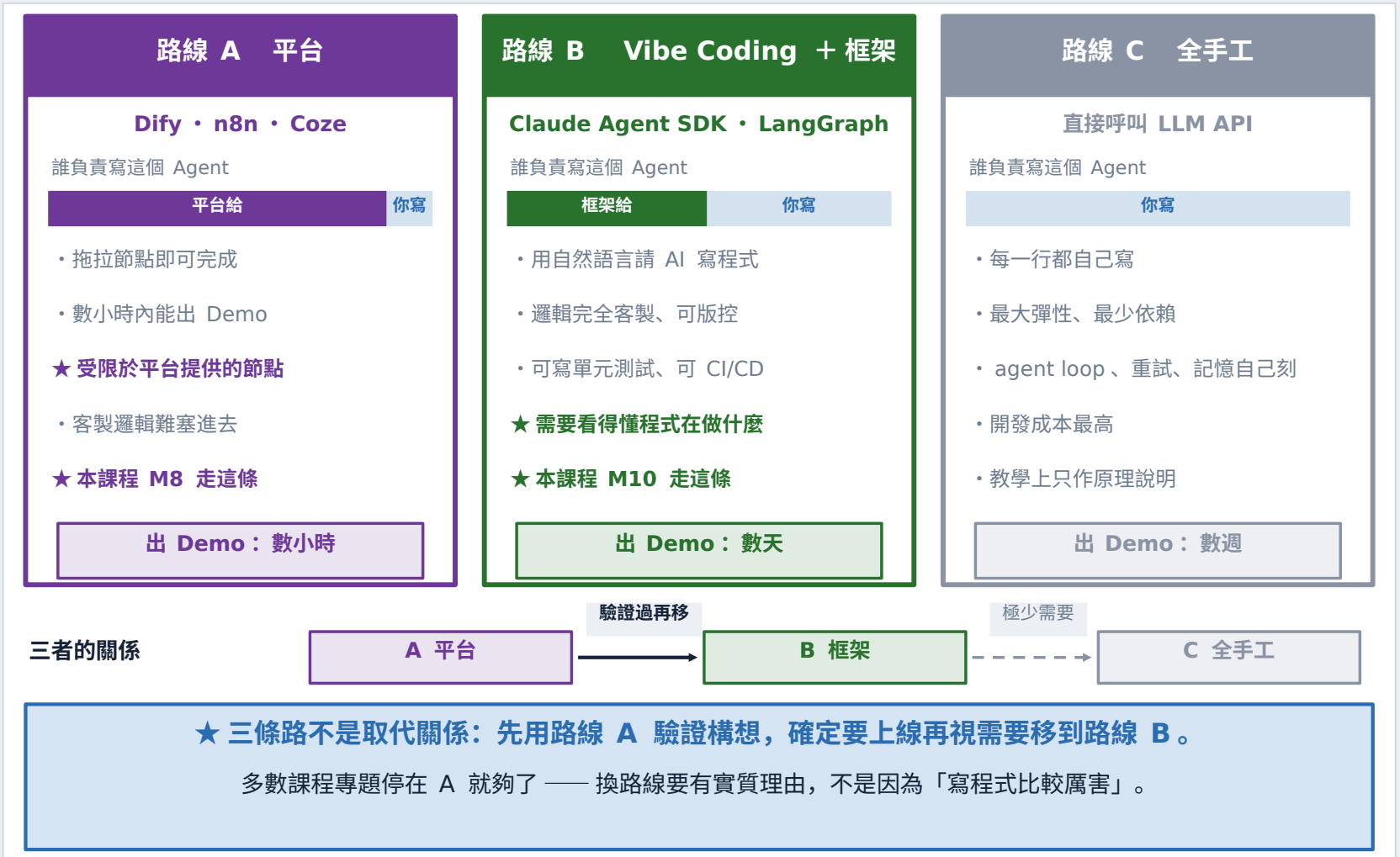
為什麼還是有門檻

- Agent 會「自己決定做什麼」，寫錯的代價比一般程式大得多
- 它可能重複呼叫付費 API、可能刪錯檔案、可能無限迴圈燒錢
- ★ Vibe Coding 讓「寫出來」變快，沒有讓「驗證」變快

本節的立場

- 先在 Dify 把流程走通 → 再決定要不要移到程式碼
- Dify 版本就是你的規格書，也是你驗證程式碼版本的對照組
- 不建議跳過 Dify 直接寫程式：你會分不清是邏輯錯還是程式錯

做一個 Agent，有三條路



路線 A 平台

Dify 、 n8n 、 Coze 。拖拉即可，數小時出 Demo ，
受限於平台提供的節點。

路線 B Vibe Coding ＋框架

請 AI 用 Claude Agent SDK 或 LangGraph 寫。
邏輯完全客製、可版控、可寫測試。

路線 C 全手工

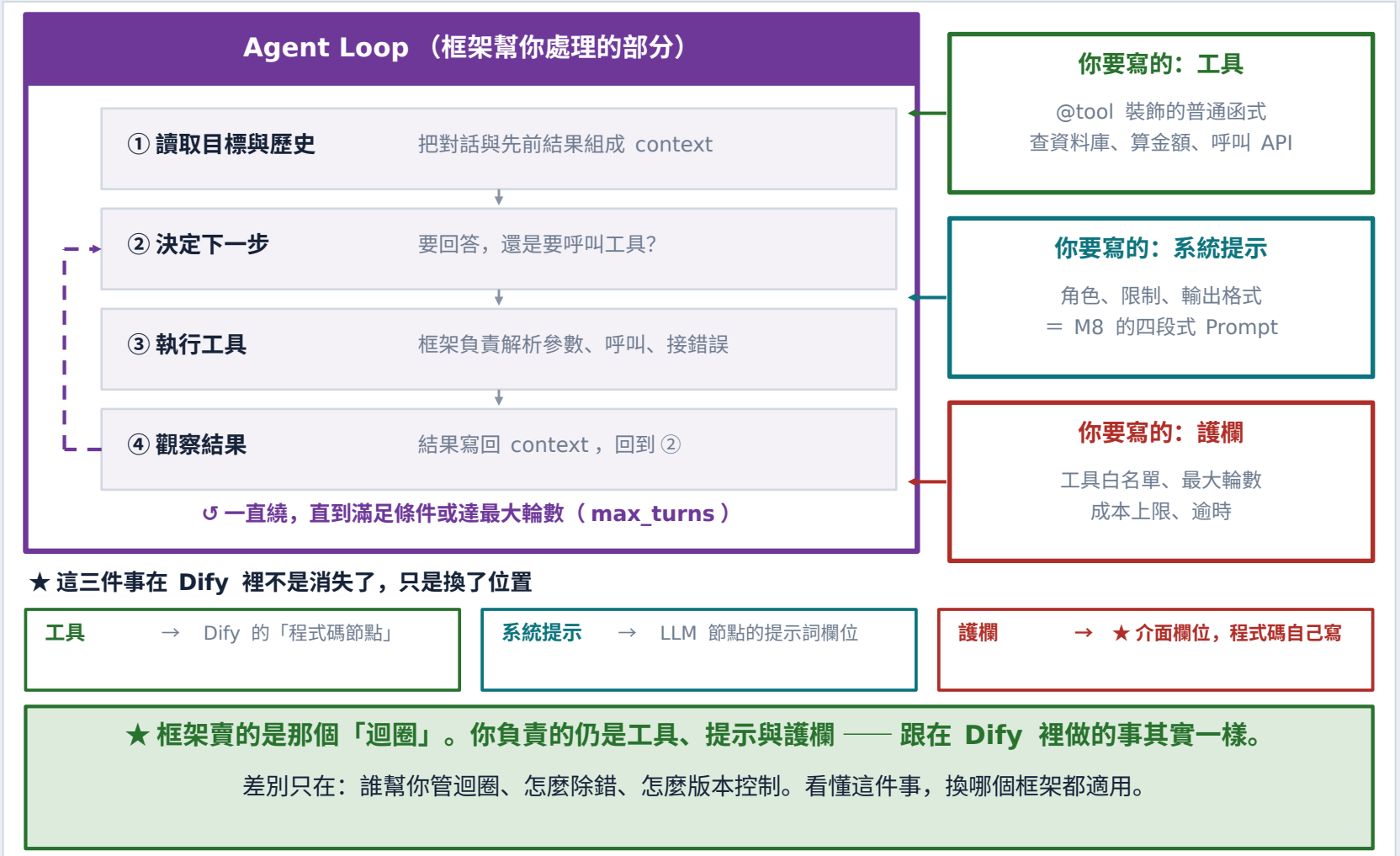
直接呼叫 LLM API ， agent loop 、重試、記憶全部自己刻。
教學上只作原理說明。

★ 不是取代關係

先用 A 驗證構想，確定要上線再視需要移到 B 。
多數專題停在 A 就夠了。

★ 原生圖形，可直接在 PowerPoint 編輯

程式碼版 Agent：框架處理什麼，你處理什麼



框架賣的是那個迴圈

讀取 context → 決定下一步 → 執行工具 → 觀察結果 → 回到第二步。
這段是所有 Agent 框架的核心價值。

你寫工具

用裝飾器標記的普通函式。查資料庫、算金額、呼叫 API——
就是 Dify 裡的「程式碼節點」。

你寫系統提示

角色、限制、輸出格式——就是 M8 的四段式 System Prompt。

你寫護欄

可用工具白名單、最大輪數、成本上限、逾時。

★ 這在 Dify 裡是設定欄位，在程式碼裡要自己寫。

★ 原生圖形，可直接在 PowerPoint 編輯

四個主流 Agent 框架（2026 年 7 月）

框架	出品	特色	什麼時候選它
Claude Agent SDK	Anthropic	把 Claude Code 的引擎包成函式庫，內建檔案讀寫、指令執行、網頁搜尋	以 Claude 為主、想要現成完整的工具與權限機制
LangGraph	LangChain	把流程建模成有向圖，狀態可檢查點、可回溯；不綁定模型供應商	需要精細控制執行路徑、多 Agent 拓撲、混用不同模型
OpenAI Agents SDK	OpenAI	與 OpenAI 生態整合最緊、上手快	主要用 GPT 系列模型
CrewAI	開源社群	以「角色分工」組多 Agent 團隊，概念直觀	任務適合拆成幾個明確角色（研究員、審查者、撰稿者）

★ 課程不指定框架。重點是理解「框架處理迴圈、你處理工具與護欄」這個分工；另有 Google ADK、 Microsoft Agent Framework 等，分工邏輯相同。

用 Vibe Coding 做 Agent 的六個步驟



第 1 步最重要

先在 Dify 把流程走通。沒有規格就開始寫，AI 只會給你
一個看起來很像但沒人驗證過的東西。

第 3 步是分水嶺

★ 先要求 AI 寫測試案例，再寫實作。
沒有測試的 Agent 不能上線——這是本節唯一的硬規則。

第 5 步不能省

最大輪數、工具白名單、成本上限、逾時、失敗回退。
Agent 沒有護欄，就是一台會自己花錢的機器。

第 6 步是誠實檢查

看不懂的程式不要上線。請 AI 逐段解釋，不懂就追問到懂。

★ 原生圖形，可直接在 PowerPoint 編輯

實際長什麼樣：一個長照給付 Agent 的骨架

與 M8 的 Dify Chatflow 對應——同一套邏輯，換一種寫法

Python （框架語法略有差異，此處示意）

```
# ① 工具＝Dify 裡的「程式碼節點」
@tool
def lookup_payment(cms_level: str) -> dict:
    """依 CMS 等級查詢長照給付額度上限。"""
    TABLE = {"2": 10020, "3": 15460, "4": 18580, "5": 24100,
             "6": 28070, "7": 32090, "8": 36180}
    import re
    m = re.search(r"[2-8]", cms_level or "")
    if not m: return {"error": "無法判定等級"}
    return {"level": m.group(), "amount": TABLE[m.group()]}

@tool
def search_regulation(query: str) -> list[str]:
    """檢索長照法規知識庫。"""   # ＝Dify 的「知識檢索節點」
    return vector_store.similarity_search(query, k=5)

# ② 系統提示＝Dify 的 System Prompt（四段式，內容完全一樣）
SYSTEM = open('prompts/ltc_system.md').read()

# ③ 護欄＝Dify 裡是設定欄位，這裡要自己寫
agent = Agent(
    system_prompt = SYSTEM,
    tools         = [lookup_payment, search_regulation],  # 工具白名單
    max_turns     = 8,          # 最大輪數，防無限迴圈
    timeout       = 60,         # 逾時
)
```

同一套邏輯，兩種寫法的對照

同一個長照給付 Agent ：每個 Dify 元件在程式碼裡是什麼

左： Dify Chatflow （ M8 做的） 右：程式碼版（本節做的）

知識檢索節點（ Top-K 、閾值）	檢索	**search_regulation() 工具函式**
LLM 節點（四段式 System Prompt ）	提示	**SYSTEM 變數，讀自同一份 .md**
程式碼節點（查表算金額）	計算	**lookup_payment() 工具函式**
模板轉換（固定輸出格式）	格式	**輸出驗證函式或 Pydantic 模型**
最大迭代次數：介面上一個欄位	迴圈	**max_turns 參數**
Tracing 面板，逐節點看	除錯	**log 、 breakpoint 、單元測試**
匯出 .yml 檔	版本	**Git ，可 diff 、可回溯**

藍框：兩邊做法一一對應。**橘框**：真正不同的地方——誰管迴圈、怎麼除錯、怎麼留版本。

Dify 版就是規格書

同一份 System Prompt 、同一張額度表，兩版問同一組題目，答案應該一樣——不一樣就是程式碼版有錯。

程式碼版要自己補的

迴圈上限、失敗處理、輸出驗證； Dify 介面上替你做好的，這裡都要寫出來。

換到程式碼的時機

需要 Dify 做不到的整合、版本控制或測試時才換（見下一頁）。

★ 看出來了嗎：兩邊做的事情一模一樣，差別在「誰幫你管迴圈」與「怎麼除錯」。

什麼時候值得從 Dify 換到程式碼

訊號	說明	建議
Dify 的節點表達不出你的邏輯	需要遞迴、複雜狀態機、動態產生流程	→ 換程式碼
需要寫自動化測試	要納入 CI/CD，每次改動都自動驗證	→ 換程式碼
需要嚴格版本控制與 code review	醫療、金融場域的稽核要求	→ 換程式碼
只是覺得「寫程式比較厲害」	沒有實質需求	→ 留在 Dify
流程還在頻繁調整	改一個節點 30 秒 vs 改程式再測 30 分鐘	→ 留在 Dify
團隊裡沒人能維護程式碼	你畢業後這個系統誰接手?	→ 留在 Dify

> 最後一列常被忽略：可維護性比技術先進度重要得多，在學校專題與醫院實務都是。

Agent 護欄清單：程式碼版一定要自己補上的六件事

護欄	為什麼需要	具體做法
① **最大輪數與逾時**	Agent 判斷「還不夠」就會再跑一輪，沒上限可能無限迴圈	同時設 max_turns 與 timeout，兩道保險
② **工具白名單與最小權限**	「先都給再說」的 Agent 可能做出你沒預期的動作	只給真正需要的工具；有寫入或刪除能力者一律要人工確認
③ **成本上限與用量記錄**	Agent 成本不像 Workflow 可預估，這是最大的財務風險	每次執行記錄 token 用量；設單次與每日上限
④ **輸出驗證與稽核軌跡**	LLM 輸出格式不保證穩定；醫療場域需可追溯	用 Pydantic 等結構化模型驗證；完整記錄每輪思考與工具呼叫
⑤ **失敗回退**	外部工具會掛掉， Agent 不會自己知道該停下來	定義降級路徑：重試上限、改用備援來源、明確回報失敗
⑥ **敏感資料過濾**	個案資料不該原樣送進外部 LLM	送出前去識別化；本地模型處理最敏感的欄位

★ 這六件事在 Dify 裡多半是介面上的設定欄位；換到程式碼，每一項都得自己寫。這是換路線的隱藏成本。

隨堂檢核｜工具選型與程式碼路線

Q 1

要做「每天早上抓昨日住院名單，用 AI 產生摘要後寄給主治醫師」，選什麼工具?

n8n 為主。需要排程觸發、讀資料庫、寄信——這三件事 n8n 都是現成節點， AI 摘要只是中間一步。若摘要需要查院內知識庫，再用 n8n 呼叫 Dify 的 API 。

Q 2

Dify 的 Agent 與程式碼寫的 Agent ，本質差別在哪?

本質沒有差別——都是「工具 + 系統提示 + 護欄 + 一個迴圈」。差別在誰幫你管迴圈、怎麼除錯、怎麼版本控制。 Dify 用介面設定，程式碼用參數與測試。

Q 3

用 Vibe Coding 寫 Agent 時，哪一步最不能省?

先寫測試（第 3 步）與加護欄（第 5 步）。 Agent 會自主決定行為，沒有測試你不知道它會做什麼，沒有護欄它可能無限迴圈或燒掉預算。

Q 4

你的專題在 Dify 做得好好的，該不該改寫成程式碼?

多數情況不該。除非 Dify 的節點表達不出邏輯、或需要自動化測試與稽核。若只是覺得寫程式比較厲害，那是換錯理由——可維護性更重要。

PART 10

課堂 Lab

Hands-on Lab

本模組沒有正式授課，但有實作任務。
做完後繳交一份自學紀錄，併入課堂參與分。

自學任務｜做一個你自己用得到的小工具

M10 自學模組

開啟檔案

原始教材（可對照更完整的指令與範例）：

`Vibe Coding - AI Driven Software Development.pptx`（37 頁，英文標題版）
`Vibe Coding - AI 驅動的軟體開發新範式.pptx`（22 頁，中文精簡版）

兩份皆位於課程根目錄。
本講義以英文標題版為基礎改寫。

預期結果

- 三到五個 Prompt 通常足以完成一個小工具
- 第一次跑起來多半不會如預期——這是正常的，記錄下來就是收穫
- 解釋不出來的段落，就是你需要補的知識點
- 工具會變，但「描述 → 驗證 → 修正」的循環不會變

操作步驟

1. 選一個你實際會用到的小需求（例如把一疊 Excel 合併成一張總表）
2. 安裝一套工具並完成設定（Cursor、Claude Code 或 Copilot 皆可）
3. 寫一份一頁的 PRD.md：目標、輸入、輸出、不做什麼
4. 建立 Git 儲存庫並完成第一次 commit
5. 用三到五個 Prompt 完成核心功能，每一步都執行確認並 commit
6. 至少經歷一次「AI 卡住 → 你用四個手法之一解決」的過程並記錄下來
7. 請 AI 逐段解釋你不懂的部分，直到你能說明整段在做什麼
8. 撰寫自學紀錄（見下頁）並上傳

自學紀錄要寫什麼（一至兩頁）

項目	內容
1　你做了什麼	工具名稱、專案目的、最後完成到什麼程度（有沒有做完都可以）
2　你用了哪些 Prompt	貼出三到五個實際使用的 Prompt。 特別標出哪一個效果好、哪一個要重寫
3　卡關與解決	描述一次 AI 卡住的情況： 它產出了什麼、為什麼不對、你怎麼讓它走回正軌
4　你看不懂的部分	誠實寫出哪一段程式碼你還是說不清楚它在做什麼。 這一項不扣分，寫得越具體越好
5　你的判斷	這個工具在你未來的工作裡，哪些地方會用、哪些地方不會用？為什麼？

評分方式：併入課堂參與分（10%）。重點在第 3、4、5 項——過程與判斷比成品重要，做不完也可以拿到分數。

課後思考題

Q1 **Vibe Coding 光譜的五個層級中，你在課堂 Lab 實際落在哪一級？為什麼停在那裡？**

提示：想想是什麼限制了你——是工具能力、你的驗證能力，還是任務的風險。

Q2 **如果一位同事用 AI 生成了一段計算長照給付金額的程式碼，你會怎麼驗證它？**

提示：想想 M1 學到的資料檢查與 M8 學到的「數字不生成、要查表」。

Q3 **AI 生成的程式碼會不會有偏誤？如果會，來自哪裡？**

提示：它學自公開的程式碼庫。那些程式碼裡內建了哪些假設？誰寫的？為誰寫的？

本模組回顧：逐條自評

1. ☐ 說出光譜五級與兩種作法的差別，並依任務風險調整 AI 的強度
2. ☐ 依形態選工具，並說明 MCP 與 M8 的工具調用是同一個概念
3. ☐ 依六步工作流與三種句型下 Prompt； AI 卡住時先給足證據
4. ☐ 從一句話做到公開網址：五行規格、迭代、本機測試、推上 GitHub 、部署
5. ☐ 辨認四類風險；說不出這段程式碼在做什麼，就不用在會影響別人的地方

★ 背景：工具會變，但六步工作流與「描述→驗證→修正」的循環不會變

下一步　第 18 週　期末報告繳交與總結評量

課後複習：本單元的互動學習站

掃描 QR code 或輸入網址；手機、平板、電腦都能用

掃我開啟 M10 互動版

手機相機對準即可開啟

網址

nchsieh.github.io/assets/courses/001/interactive/m10/

19 節，順序對應講義各 PART

看完一個 PART 就做對應的那一節，不必一次做完

改參數、立刻看結果

圖表可以拖拉調整，看懂「為什麼」，而不只是背結論

隨堂練習＋即時回饋

答錯會說明原因；作答進度只存在你自己的瀏覽器

全部單元入口： nchsieh.github.io/assets/courses/001/interactive/ · 網站內容為教學示例與模擬資料，不需登入